

TP 2 ARP, ICMP et NAT

TP Réseaux Avancés M1 SIC - IA

Ilyas Bambrik

Table des matières



Objectifs	3
I - Exercice : ARP et ICMP	4
II - Exercice : Fragmentation IP et flags	5
III - Exercice : UDP broadcast	6
IV - Exercice : NAT (Network Address Translation) [optionnel]	7

Objectifs

- Analyser les messages ARP avec Wireshark ;
- Analyser les types de messages ICMP avec Wireshark ;
- Tester la configuration de NAT ;
- Travail à rendre : pour chaque question/étape prenez une capture d'écran *complète* (les captures d'écran partielles ne seront comptabilisés). Le document à consulter doit être un fichier PDF avec les réponses et captures numérotées ;
- L'organisation du document à consulté est comptabilisée dans la note ;
- Le TP sera consulté deux semaines après la publication de celui-ci ;

Les codes présentés dans ce support sont téléchargeables à partir du lien suivant :
<https://drive.google.com/drive/u/1/folders/1G6y93EPFtjf8jkyw1DhE6mHUJO0wX1GE>

Exercice : ARP et ICMP



Question

Pour les étudiants souhaitant de tester ce TP à domicile, il suffit de connecter au réseau domestique et puis utiliser des adresses IP du réseaux locale (192.168.1.1--192.168.1.254). Vous pouvez aussi utiliser le point d'accès d'un Smartphone pour créer un réseau local :

Ouvrez votre CMD et Wireshark. Ensuite, ouvrez l'interface réseau que vous utilisez pour connecter au réseau sur Wireshark (l'interface Wifi si vous êtes connecté au réseau local ou réseau de la faculté).

1. Connectez au réseau local et repérez les messages ARP diffusés dans le réseau (ajoutez un filtre *arp.opcode==2 || arp.opcode==1*). Pour cette étape vous devez capturer l'interface Wifi ou Ethernet (les messages arp ne seront pas reçus par l'interface loopback);
2. Quels sont les types des messages ARP affichés après l'application du filtre ? Repérez votre adresse MAC dans les messages ARP?
3. Exécutez la commande *ping* sur votre CMD vers une machine distante (ou vers le loopback *127.0.0.1*) et repérez les paquets ICMP Echo request / reply dans Wireshark. Pour repérer les messages ICMP echo request/reply, ajoutez le filtre *icmp.type == 0 || icmp.type==8* dans la capture de paquets Wireshark.
4. Sélectionnez un paquet ICMP echo request/reply et repérez le payload (champ Data du paquet ICMP) généré par le ping dans le paquet affiché par Wireshark.
5. Exécutez la commande *tracert* sur votre CMD vers une machine distante (ou site web par exemple *google.dz*) et ajoutez le filtre *icmp.type == 0 || icmp.type==8 || icmp.type==11*. Quels sont les types de messages ICMP générés par la commande *tracert* ?
6. Repérez la copie du message expiré dans le message ICMP type 11 (*Time Exceeded*) .

Remarque : La commande traceroute de linux n'utilise pas ICMP Echo Request et ICMP Echo/Reply. Celle-ci génère des Datagrames UDP vers la destination avec le même principe de TTL incrémenté que *tracert*. Ainsi, les sauts intermédiaires répondent avec `icmp.type==11` et `icmp.type==3` (si vous utilisez Linux vous devez ajouter le filtre `icmp.type == 3 || icmp.type==11`).

Exercice : Fragmentation IP et flags

II

Question

1. Sélectionnez l'interface Wifi et exécutez le programme python suivant. Celui-ci transmet un message UDP vers le port 50000 d'une machine locale (et cette dernière ne possède pas de programme écoutant ce port). Ainsi, le contenu ne peut pas être livré.
2. Quel est le type du message ICMP et le code reçu à cause de cette transmission (capturez la transmission avec Wireshark) ?
3. Repérez la copie du message qui n'a pas pu être livrer au port 50000 dans le message ICMP.
4. Changez l'adresse IP destination dans le programme suivant en mettant `IP="172.16.160.2"` (ou une adresse de votre choix dans le réseau local par exemple 192.168.1.1 si vous êtes connectés au réseau local chez vous) et `MESSAGE="ABC"*1500`. (`MESSAGE="ABC"*1500` == une chaîne de caractères de 4500 (3 x 1500) où "ABC" se répète pour 1500 fois)
5. Utilisez le filtre (`ip.frag_offset!=0 || ip.flags.mf==1`) dans Wireshark pour repérer la transmission des fragments ;
6. Exécutez le code python et repérez les flags allumés dans le header IP pour indiquer la fragmentation.

```

1 import socket
2 IP = "192.168.1.1"
3 PORT = 50000
4 MESSAGE = "Ey 192.168.1.1! tu ecoute le port 50000?"
5 print( "UDP target IP:", IP)
6 print( "UDP target port:", PORT)
7 print( "message:", MESSAGE)
8 #creation d'un socket UDP
9 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
10 sock.sendto(MESSAGE.encode(), (IP, PORT))
11

```

Exercice : UDP broadcast

III

Dans cet exercice, vous devez prendre une capture d'écran complète de votre écran d'un broadcast UDP sur le port 5000.

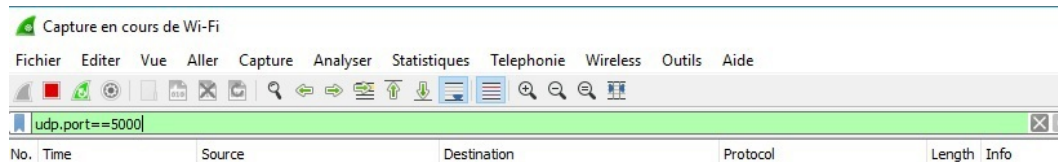


Figure 4. Filtre UDP

Question

UDP est moins utilisé que TCP mais présente d'autres avantages. Par exemple, puisque le TWH n'est pas nécessaire dans UDP, une application peut diffuser périodiquement sur un numéro de port spécifique afin que d'autres programmes/appareils puissent détecter l'existence de celle-ci dans le réseaux local. C'est le cas des jeux vidéos multijoueur dans le réseau local.

1. Connectez au réseau local.
2. Lancez Wireshark *sur la machine de votre camarade* ou d'une machine quelconque dans le réseau local (votre camarade doit être connecté au même réseau local aussi).
3. Filtrez pour obtenir les messages UDP entrants du port 5000 sur la machine de votre camarade (*udp.port==5000*);
4. Exécutez le *programme suivant sur votre machine* afin de diffusez un message dans le réseau (le code est téléchargeable depuis le lien suivant <https://drive.google.com/drive/folders/1jgkUyXKXEsXb-3nhJT-gDt2GRHmZelNh?usp=sharing>) :
5. Vous devez modifier l'adresse contenue dans la variable *AdresseDiffusion* avec votre propre adresse de diffusion. Pour trouver celle-ci vous pouvez utiliser l'une des deux commandes :
 - *arp -a* puis trouver l'adresse IP correspondant l'adresse mac de diffusion (ff :ff :ff :ff :ff :ff) ;
 - *piconfig* puis trouvez l'adresse IP de votre interface et masque afin d'appliquer ce dernier et trouver l'adresse IP de broadcast ;

```
1 import socket
2 SocketUDP= socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # creation socket UDP
3 SocketUDP.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1) # rendre le
  socket UDP broadcast
4 Numero_Port=5000
5 AdresseDiffusion='192.168.1.255'
6 MessageADiffuser=b'Votre message a diffuser dans le reseau' # contenu du message
  diffusion
7 SocketUDP.sendto(MessageADiffuser, (AdresseDiffusion, Numero_Port)) #
  transmettre le message
```

Exercice : NAT (Network Address Translation) [optionnel]

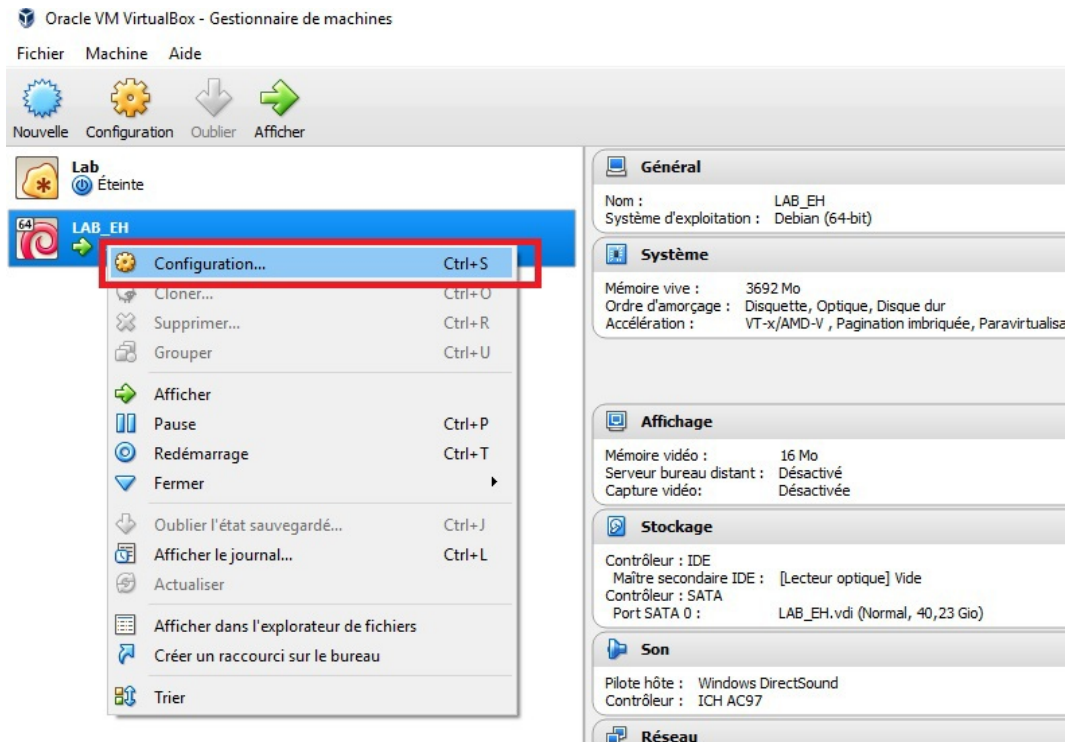
IV

Cet exercice nécessite une présentation directe du travail en exécution et non pas des captures d'écran :

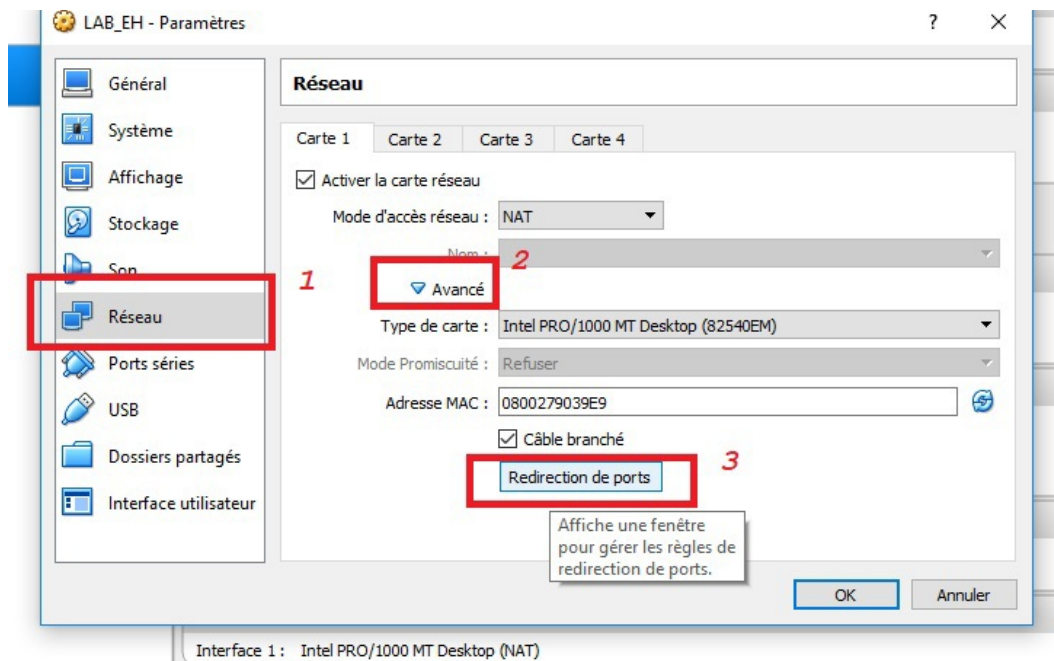
- Téléchargez Oracle VirtualBox et Oracle VirtualBox Extension Pack ;
- Installez Oracle VirtualBox en premier et ensuite installez VirtualBox Extension Pack (double clique sur le cube vert après la fin de l'installation de Oracle VirtualBox) ;
- Double clique sur *Lubuntu+wireshark+Idle.ova* (cube orange) et définissez la configuration qui vous convient (RAM, processeur, etc). Pour une installation avec configuration par défaut, cliquez seulement sur suivant jusqu' à la fin de l'installation (téléchargez l'image de la machine virtuelle dans le lien <https://drive.google.com/drive/folders/1G6y93EPFtjf8jkyw1DhE6mHUJO0wX1GE?usp=sharing>) ;

- [5.2 SDK \(5.2.24\)](#)
- **VirtualBox 5.2.24 (released January 15 2019)**
 - [Windows hosts](#)
 - [OS X hosts](#)
 - [Solaris hosts](#)
 - Linux Hosts:
 - [Ubuntu 18.04 / 18.10 / Debian 10](#)
 - [Ubuntu 16.04](#) [32-bit](#) | [64-bit](#)
 - [Ubuntu 14.04 / 14.10 / 15.04](#) [32-bit](#) | [64-bit](#)
 - [Debian 9](#) [32-bit](#) | [64-bit](#)
 - [Debian 8](#) [32-bit](#) | [64-bit](#)
 - [openSUSE 15.0](#)
 - [openSUSE 13.2 / Leap 42](#) [32-bit](#) | [64-bit](#)
 - [Fedora 29](#)
 - [Fedora 26 / 27 / 28](#) [32-bit](#) | [64-bit](#)
 - [Oracle Linux 7 / Red Hat Enterprise Linux 7 / CentOS 7](#)
 - [Oracle Linux 6 / Red Hat Enterprise Linux 6 / CentOS 6](#) [32-bit](#) | [64-bit](#)
 - [All distributions](#) [32-bit](#) [64-bit](#)
 - [Extension Pack](#)
 - [Sources](#)
 - [MD5 checksums, SHA256 checksums](#)

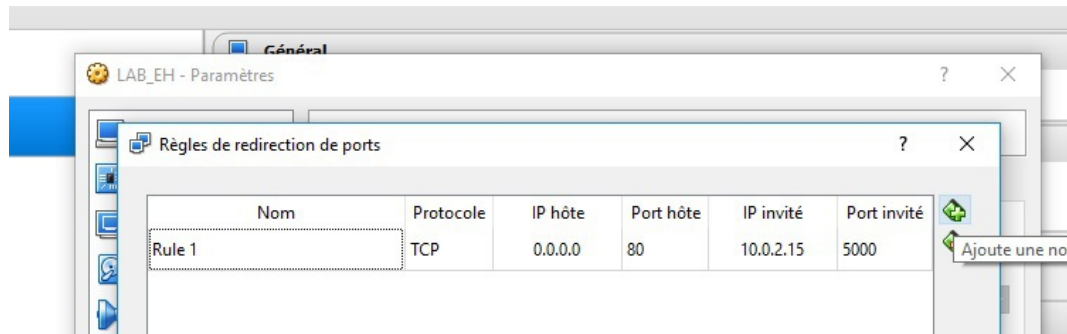
Après la fin de l'installation de la machine virtuelle , cliquez sur celle-ci et ensuite cliquez sur "Configuration..."



Cliquez sur *Réseau*, ensuite *Avancé* et ensuite sur *Redirection de Ports*.



Ajoutez l'entree suivante et puis cliquez sur *OK* :



Lancez votre machine virtuelle (login == *master1rsd2020*, mot de passe == *master1rsd2020*) ;

Exécutez le programme suivant dans *votre machine virtuelle* (ouvrez le terminal et tapez "*python NomdeVotreProgramme.py*") ;

```
1 import socket
2 SocketServeur=socket.socket()
3 SocketServeur.bind(("0.0.0.0", 5000))
4 SocketServeur.listen(4)
5 while True:
6     ConnexionAuClient, addr = SocketServeur.accept()
7     print(ConnexionAuClient, addr)
8     print(ConnexionAuClient.recv(1024))
9     ConnexionAuClient.send(b"\r\n\r\n<html><body><h1>FAKEHTMLSERVER INC &#xa9;<
    /h1></body></html>")
10    ConnexionAuClient.close()
```

Dans votre système d'exploitation principale ouvrez votre navigateur et tapez url suivant : *http://127.0.0.1/*

Le site est accessible de toutes les machines du réseau local (*http://VOTREADRESSEIP/* depuis n'importe quelle machine) à condition que le programme serveur python est lancé.

